TL;DR

Prompt: TL;DR: Explain e-commerce E2E automation flow using playwright in detail
Output: TL;DR:
An e-commerce E2E automation flow using Playwright automates and validates the complete customer purchase journey to ensure critical business workflows function correctly across browsers and environments.

Typical E2E flow includes:

1. Launch browser and navigate to the application
2. Login with valid user credentials
3. Search for a product using search/filter functionality
4. Open product details page
5. Add product to cart
6. Validate cart items, quantity, and pricing
7. Proceed to checkout
8. Enter shipping and payment details
9. Place the order
10. Verify order confirmation message/order ID
11. Logout and close browser

Using Playwright, QA engineers implement:

- Page Object Model (POM) for reusable code
- Locators and assertions for validations
- Auto-waiting to handle dynamic UI
- Data-driven testing for multiple scenarios
- Screenshots/videos for failed tests
- Cross-browser execution (Chrome, Edge, Firefox, WebKit)
- API/network interception for backend validation
- CI/CD integration for automated regression execution

This helps ensure stable releases, faster regression cycles, reduced manual effort, and reliable validation of critical e-commerce workflows.

ELI 15

Prompt: ELI 15 Explain e-commerce E2E automation flow using playwright in detail
Output: Imagine an e-commerce website like Amazon.
E2E (End-to-End) automation means testing the complete shopping journey automatically, just like a real customer would use the website.

Playwright is a tool that acts like a robot user inside the browser.

Simple E-commerce E2E Flow Using Playwright

Step 1: Open the Website

Playwright launches the browser and opens the shopping website.

Example:

- Open Chrome
- Go to www.shoppingdemo.com

Step 2: Login

The automation enters:

- Username
- Password
- Clicks Login button

Then it checks:
✅ Did login succeed?
✅ Did dashboard/homepage load?

Step 3: Search for Product

The script types:

"iPhone"

inside the search bar.

Then it verifies:
✅ Search results are displayed
✅ Correct products appear

Step 4: Open Product Page

Automation clicks one product.

Then checks:
✅ Product title
✅ Price
✅ Images
✅ Stock availability

Step 5: Add Product to Cart

Playwright clicks:

“Add to Cart”

Then validates:
✅ Cart count increased
✅ Correct item added
✅ Price is correct

Step 6: Checkout Process

Automation clicks:

“Proceed to Checkout”

Then fills:

- Address
- Phone number
- Delivery details

Step 7: Payment

The script enters test payment details.

Example:

- Card number
- CVV
- OTP (mock/test)

Then clicks:

“Pay Now”

Step 8: Order Confirmation

Finally, automation checks:
✅ “Order Placed Successfully” message
✅ Order ID generated
✅ Confirmation page loaded

This is the most important validation because it confirms the entire flow works correctly.

Automation Framework Architecture

Context:

Act as an automation architect to design playwright framework with bdd cucumber for Healthcare application

Information:

[MANDATORY] Ask me 5 valid multiple choice question one by one

[STRICT] Generate next question only the first question is answered

[MANDATORY] Generate the automation framework only when all the questions are answered.

[MANDATORY] Include API, Reports , screenshots.

[CRITICAL] Do not skip steps

[STRICT] Review the framework architecture as a manager.

Question 1/5

What type of Healthcare application should the automation framework support?

A. Patient Management System
B. Electronic Health Records (EHR/EMR)
C. Healthcare Insurance Portal
D. Telemedicine Application
E. Multi-module Healthcare SaaS Platform

Please choose one option (A–E).

A

Question 2/5

What level of automation coverage is expected in the framework?

A. UI Automation only
B. API Automation only
C. UI + API Automation
D. UI + API + Database Validation
E. Complete End-to-End Integration Automation

Please choose one option (A–E).

C

Question 3/5

Which reporting strategy should the framework include?

A. Basic HTML Reports
B. HTML Reports + Screenshots
C. Allure Reports + Screenshots
D. Allure Reports + Screenshots + Videos
E. Enterprise Dashboard Reporting with historical trends

Please choose one option (A–E).

C

Question 4/5

Which execution strategy should the framework support?

A. Sequential execution only
B. Parallel browser execution
C. Tag-based execution with parallel support
D. Distributed execution across environments
E. Cloud/Grid execution using BrowserStack or SauceLabs

Please choose one option (A–E).

C

Question 5/5

Which framework architecture pattern do you prefer?

A. Simple folder-based framework
B. Page Object Model (POM)
C. POM with reusable utilities
D. Layered enterprise framework
E. Scalable hybrid enterprise framework with reusable components

Please choose one option (A–E).

E

Enterprise Playwright + Cucumber BDD Framework

Healthcare Application — Patient Management System

Selected Requirements

Area	Selection
Application Type	Patient Management System
Automation Scope	UI + API Automation
Reporting	Allure Reports + Screenshots

Area	Selection
Execution Strategy	Tag-based Parallel Execution
Framework Architecture	Scalable Hybrid Enterprise Framework

Recommended Enterprise Framework Architecture

```
Healthcare-Automation-Framework
│
├── .github/
│   └── workflows/
│       └── playwright-ci.yml
│
├── config/
│   ├── qa.env
│   ├── staging.env
│   └── framework.config.ts
│
├── src/
│   │
│   ├── features/
│   │   ├── login.feature
│   │   ├── patientRegistration.feature
│   │   ├── appointment.feature
│   │   └── billing.feature
│   │
│   ├── step-definitions/
│   │   ├── login.steps.ts
│   │   ├── patient.steps.ts
│   │   ├── appointment.steps.ts
│   │   └── billing.steps.ts
│   │
│   ├── pages/
│   │   ├── LoginPage.ts
│   │   ├── DashboardPage.ts
│   │   ├── PatientPage.ts
│   │   ├── AppointmentPage.ts
│   │   └── BillingPage.ts
│   │
│   ├── api/
│   │   ├── authAPI.ts
│   │   ├── patientAPI.ts
│   │   └── appointmentAPI.ts
│   │
│   ├── hooks/
```

```
│   │   ├── beforeHooks.ts
│   │   └── afterHooks.ts
│   │
│   ├── fixtures/
│   │   └── baseFixture.ts
│   │
│   ├── utils/
│   │   ├── logger.ts
│   │   ├── screenshotHelper.ts
│   │   ├── retryHelper.ts
│   │   ├── waitHelper.ts
│   │   └── environmentHelper.ts
│   │
│   ├── validations/
│   │   ├── uiAssertions.ts
│   │   └── apiAssertions.ts
│   │
│   ├── test-data/
│   │   ├── qa/
│   │   └── staging/
│   │
│   └── constants/
│       └── routes.ts
│
├── screenshots/
├── allure-results/
├── allure-report/
├── playwright-report/
│
├── cucumber.js
├── playwright.config.ts
├── tsconfig.json
└── package.json
```

Framework Design Approach

This framework follows:

- Playwright + TypeScript
- Cucumber BDD
- Page Object Model (POM)
- Hybrid enterprise architecture
- API integration support
- Parallel execution strategy

The design focuses on:
✅ Scalability
✅ Maintainability
✅ Reusability
✅ Enterprise readiness

---

1. Cucumber BDD Integration

Sample Feature File

```
Feature: Patient Registration

  @smoke @patient
  Scenario: Register new patient successfully

    Given user launches healthcare application
    When user logs into the application
    And user registers a new patient
    Then patient should be created successfully
```

Benefits:

- Easy collaboration with business teams
- Readable scenarios
- Better traceability

---

2. Step Definition Layer

Sample Step Definition

```
Given('user launches healthcare application', async function () {
  await page.goto(process.env.BASE_URL);
});

When('user logs into the application', async function () {
  await loginPage.login('admin', 'password');
});

Then('patient should be created successfully', async function () {
  await expect(patientPage.successMessage).toBeVisible();
});
```

Purpose:
✅ Maps feature files to automation code
✅ Supports reusable steps

---

3. Page Object Model (POM)

Sample Page Object

```
export class LoginPage {

  async login(username: string, password: string) {
      await this.username.fill(username);
      await this.password.fill(password);
      await this.loginButton.click();
  }
}
```

Benefits:
✅ Centralized locators
✅ Easier maintenance
✅ Cleaner test code

---

4. API Integration Layer

Since UI + API automation was selected, framework includes API helpers.

Sample API Utility

```
import { request } from '@playwright/test';

export async function getPatientDetails(patientId: string) {

   const apiContext = await request.newContext();

   const response = await apiContext.get(
      `/api/patient/${patientId}`
   );

   return response;
}
```

API validations supported:

- Status code validation
- Response payload validation
- Backend verification after UI actions

---

5. Reporting Strategy

Included Reports

✅ Allure Reports
✅ Playwright HTML Reports
✅ Failure screenshots

Screenshot Strategy

On test failure:

- Screenshot captured automatically
- Attached to Allure report

Example:

```
await page.screenshot({
  path: `screenshots/${Date.now()}.png`
});
```

Benefits:
✅ Faster debugging
✅ Easier RCA (Root Cause Analysis)

---

6. Hooks Implementation

Before Hooks

- Browser initialization
- Environment setup
- Login setup

After Hooks

- Screenshot on failure
- Cleanup
- Report attachment

Example:

```
After(async function (scenario) {

  if (scenario.result?.status === 'FAILED') {
    await page.screenshot({
      path: `screenshots/failure.png`
    });
```

```
  }
});
```

---

7. Parallel Execution Strategy

Selected:

Tag-based execution with parallel support

Example:

npx cucumber-js --tags "@smoke"

Supported tags:

@smoke
@regression
@patient
@billing

Benefits:
✅ Faster execution
✅ Selective regression runs
✅ CI/CD optimization

---

8. Environment Configuration

Separate env files:

qa.env
staging.env

Purpose:
✅ Avoid hardcoded values
✅ Environment-specific execution

---

9. CI/CD Integration

GitHub Actions Pipeline

```
name: Healthcare Automation

on:
  push:
    branches: [main]

jobs:
  test:
```

```
  runs-on: ubuntu-latest

  steps:
    - uses: actions/checkout@v3

    - name: Install dependencies
      run: npm install

    - name: Run tests
      run: npx playwright test

    - name: Generate Allure Report
      run: allure generate allure-results
```

Benefits:
✅ Automated regression
✅ Faster feedback
✅ Continuous validation

---

10. Reporting & Failure Handling Flow

Test Execution
↓
Failure Occurs
↓
Screenshot Captured
↓
Logs Attached
↓
Allure Report Generated
↓
CI/CD Publishes Report

Test Strategy Document

Context:
Act as an Experienced QA Tester and design a test strategy for Health care Patient Management System

Introduction:

[STRICT] Ask 5 valid multichoice question one by one
[MANDATORY] Once after answering all the question go to the next question
[MANDATORY] Generate the document step by step

[CRITICAL] Do no repeat any steps again
[HIGH PRIORITY] Include CI/CD approach

Example:
- Definition of each section
- Real-time Healthcare application examples
- Sample entries for each section
- Testing types
- Automation strategy
- API testing strategy
- Risks and mitigation
- CI/CD approach

Output:
Generate the document as docx and downloadable

Review the above document as Test Lead

Question 1/5

What type of testing coverage should the Test Strategy primarily focus on for the Healthcare Patient Management System?

A. Functional Testing only
B. Functional + Regression Testing
C. Functional + API + Integration Testing
D. End-to-End Testing with Automation
E. Complete QA Strategy including Manual, Automation, API, Security, and CI/CD

Please choose one option (A–E).

A

Question 2/5

Which test execution approach should the Test Strategy follow?

A. Manual testing only
B. Manual + Smoke automation
C. Manual + Regression automation
D. Hybrid approach with Manual, API, and Automation testing
E. Fully automated CI/CD-driven testing

Please choose one option (A–E).

A

Question 3/5

Which environments should be included in the Test Strategy?

A. QA only
B. QA + Staging
C. QA + Staging + UAT
D. QA + Staging + Production Smoke Validation
E. Multi-environment enterprise setup

Please choose one option (A–E).

C

Question 4/5

Which reporting and defect management approach should the Test Strategy include?

A. Excel-based reporting only
B. Jira defect tracking with manual reports
C. Jira + Test execution reports
D. Allure/HTML reports with Jira integration
E. Enterprise dashboard reporting with metrics and analytics

Please choose one option (A–E).

C

Question 5/5

Which CI/CD approach should be included in the Test Strategy?

A. No CI/CD integration
B. Basic nightly execution
C. GitHub Actions-based automated test execution
D. Jenkins pipeline with automated regression
E. Enterprise CI/CD with parallel execution and release gates

Please choose one option (A–E).

B

Your document has been generated successfully.

Download the Healthcare Test Strategy Document

Test Strategy Document - Healthcare Patient Management System

This Test Strategy document defines the overall testing approach for the Healthcare Patient Management System.

1. Project Overview

The application manages patient registration, appointments, billing, and healthcare workflows.

2. Scope of Testing

In Scope:
- Login

- Patient registration
- Appointment booking
- Billing workflows

Out of Scope:
- Third-party integrations
- Production testing

3. Test Objectives

Validate functional workflows and ensure business requirements are met.

4. Testing Types

- Functional Testing
- Smoke Testing
- Regression Testing
- Integration Testing
- UAT

5. Test Environment

- QA
- Staging
- UAT

Browsers:
- Chrome
- Firefox
- Edge

6. Test Execution Approach

Manual testing only with exploratory and functional validation.

7. Defect Management

Tool: Jira
Workflow:
Open → Assigned → Fixed → Retest → Closed

8. API Testing Strategy

Basic API validation for critical workflows like patient registration and appointments.

9. Automation Strategy

Future scope includes smoke and regression automation.

10. Risks and Mitigation

Environment instability → Backup environment
Delayed builds → Buffer testing cycle

11. CI/CD Approach

Nightly execution strategy with automated smoke validation after deployment.

12. Entry and Exit Criteria

Entry: Stable build and environment ready.
Exit: Critical defects closed and regression completed.

13. Deliverables

- Test cases
- Defect reports
- Test summary reports

14. Test Lead Review

Strengths:
- Clear scope
- Structured testing approach
- CI/CD support

Improvements:
- Add automation coverage
- Enhance API testing